

I. Python Code

```
1 import cv2 as cv
2 import numpy as np
3 import os
4 import requests
5 #連線到ESP32-CAM ip
6 url="http://172.20.10.11:81/stream"
7 stream=requests.get(url,stream=True)
8
9 #初始化照片計數器
10 photo_counter=0
11
12 #確保儲存圖片的目標資料夾存在
13 save_dir="C:/Users/Asus/Desktop/CoffeeBeans/"
14 os.makedirs(save_dir,exist_ok=True)
15
16 #定義淺、中、灰顏色區間 (HSV)
17 lower_light=np.array([10,30,90])#HSV淺色範圍
18 upper_light=np.array([30,150,200])
19
20 lower_medium=np.array([10,50,40])#HSV中等顏色範圍
21 upper_medium=np.array([35,200,80])
22
23 lower_dark=np.array([5,50,30])#HSV深色範圍
24 upper_dark=np.array([35,255,50])
25
26 #定義濾除背景區間
27 lower_brown=np.array([3,20,25])#下限
28 upper_brown=np.array([35,180,160])#
29
30 for chunk in stream.iter_content(chunk_size=8192):
31     #找到 JPEG 影像的頭部和尾部
32     jpghead=chunk.find(b'\xff\xd8')
33     jpgend=chunk.find(b'\xff\xd9')
34     if jpghead!=-1 and jpgend!=-1:
35         img=chunk[jpghead:jpgend+2]
36         frame=cv.imdecode(np.frombuffer(img,np.uint8),cv.IMREAD_COLOR)
```

引入函式庫

1. CV2：處理影像和進行計算機視覺相關的操作。
2. Numpy：用於定義 HSV 範圍和處理影像資料。
3. OS：用於檔案和目錄操作，建立截圖存檔目標資料夾。
4. Requests：從 ESP32-CAM 的 HTTP 串流讀取影像資料。

連結到 ESP32-CAM 並獲取影像資料

設定 HSV 值深、淺、中定義範圍
藉由大量樣本測試找出適合的值

從串流中提取影像

1. Chunk：從 HTTP 連線逐段讀取的資料塊，並每次讀取 8K Byte 的資料。
2. 找到 JPEG 的頭和尾：如果頭尾標記都存在，則可以提取這幀影像在 img。
3. cv.imdecode：將 img 的 JPEG 數據的二進位資料，進一步解碼為 OpenCV 可以處理的影像格式（BGR 圖像）。

- RGB 與 BGR：皆是指 RED、GREEN、BLUE，差別在於顏色資料排列順序，OpenCV 是基於早期硬體的資料格式，當時許多硬體處理影像時以 BGR 儲存顏色資料。
- JPEG 檔案結構：JFIF，程式碼中藉由找到頭(FFD8)與尾(FFD9)，判定檔案存在。

JFIF檔案結構		
段	代碼	描述
SOI	FF D8	圖像開始 (Start of Image)
JFIF-APP0	FF E0 s1 s2 4A 46 49 46 00 . . .	見下
JFXX-APP0	FF E0 s1 s2 4A 46 58 58 00 . . .	可選，見下
... 額外標記段 (例如SOF、DHT、COM)		
SOS	FF DA	掃描開始 (Start of Scan)
壓縮的圖像資料		
EOI	FF D9	圖像結束 (End of Image)

表 4.1 JFIF 檔案結構

- RGB 值會受照明環境的影響：只要光源的顏色或強度發生變化，RGB 三個資料的值都有可能改變，造成顏色誤判。
- HSV 色彩空間：是一種基於人類視覺直覺的顏色表示模型，常用於圖像處理和計算機視覺領域。HSV 的全稱是 Hue（色相）、Saturation（飽和度）和 Value（明度）。進行顏色辨識時，特別是在光源不穩定的環境中，HSV 色彩模型變得非常有用。

```

37 if frame is not None:
38     #進行高斯模糊處理
39     blurred_frame=cv.GaussianBlur(frame,(5,5),0)
40
41     #轉換為HSV顏色空間
42     hsv_image=cv.cvtColor(blurred_frame,cv.COLOR_BGR2HSV)
43
44     #建立咖啡豆範圍的遮罩
45     mask=cv.inRange(hsv_image,lower_brown,upper_brown)
46     #篩選影像
47     filtered_result=cv.bitwise_and(blurred_frame,blurred_frame,mask=mask)

```

將影像進行處理

1. cv.GaussianBlur：進行高斯模糊，平滑影像，去除雜訊。
2. cv.cvtColor：將影像進行顏色空間的轉換（例如從 BGR 轉換為 HSV）。

建立遮罩顯示濾除背景的畫面

1. cv.inRange：根據指定範圍（HSV）建立遮罩（只保留範圍內的像素）。
2. cv.bitwise_and：「按位 AND」操作的函式，將兩張影像的像素值逐一進行按位比較，並在遮罩（mask）的範圍內保留符合條件的結果。

```

49 #在影像中央畫一個十字定位點
50 height,width,_=frame.shape
51 center_x=center_y=width//2,height//2
52 line_color=(0,255,0)#十字架顏色(綠色)
53 line_thickness=2
54 #在原始影像畫十字架
55 cv.line(frame,(center_x-10,center_y),(center_x+10,center_y),line_color,line_thickness)
56 cv.line(frame,(center_x,center_y-10),(center_x,center_y+10),line_color,line_thickness)
57 #在篩選後的影像畫十字架
58 cv.line(filtered_result,(center_x-10,center_y),(center_x+10,center_y),line_color,line_thickness)
59 cv.line(filtered_result,(center_x,center_y-10),(center_x,center_y+10),line_color,line_thickness)

```

在影像中央畫十字架以定位

輪廓檢測

1. cv.findContours：函式找到影像中的所有輪廓。
2. cv.RETR_EXTERNAL：只檢測外層輪廓，不考慮內部的嵌套結構。
3. cv.CHAIN_APPROX_SIMPLE：壓縮輪廓，僅保留拐點，減少冗餘點數據。
4. contours：找到的輪廓集合，每個輪廓是一組點的座標陣列。

```

62 contours,_=cv.findContours(mask,cv.RETR_EXTERNAL,cv.CHAIN_APPROX_SIMPLE)
63 nas_corner=True#假設沒有缺角
64
65 for contour in contours:
66     #計算輪廓的凸包
67     convex_hull=cv.convexHull(contour)
68
69     #計算輪廓的面積與凸包面積
70     contour_area=cv.contourArea(contour)
71     convex_hull_area=cv.contourArea(convex_hull)
72
73     #計算輪廓面積與凸包面積的比值
74     convexity_ratio=contour_area/convex_hull_area if convex_hull_area!=0 else 1
75     cv.drawContours(frame,[convex_hull],-1,(0,0,255),2)#紅色框出缺角

```

比較物體輪廓的面積與其凸包，檢測是否有缺角

1. cv.convexHull(contour)：計算給定輪廓的凸包。
2. 凸包：包含輪廓所有點的最小凸多邊形。
3. cv.contourArea()：計算凸包與輪廓的面積。
4. convexity_ratio = contour_area / convex_hull_area。
比值接近 1：輪廓與凸包的面積接近，物體完整，沒有缺角。
比值遠小於 1：輪廓面積遠小於凸包面積，物體可能有缺角。
5. cv.drawContours()：在影像上將用紅色標記出凸包。

```

75 #顯示篩選後的影像
76 cv.imshow("Filtered Coffee Beans",filtered_result)
77 #顯示原始影像
78 cv.imshow("ESP32-CAM",frame)

```

將影像以視窗顯示

1. cv.imshow()：在視窗中顯示影像原始影像與濾除背景後的影像。

```

80     key=cv.waitKey(1)&0xFF
81     if key==ord('q'):#按下 'q' 結束程式
82         break
83     elif key==ord('a'):#按下 'a' 拍照並儲存圖片
84         photo_counter+=1
85         photo_path=os.path.join(save_dir,f"coffee_photo_{photo_counter}.jpg")
86         if not cv.imwrite(photo_path,frame):
87             print(f"無法儲存圖片至:{photo_path}")
88         else:
89             print(f"照片已儲存至:{photo_path}")

```

儲存影像截圖與關閉程式

```

90     elif key==ord('s'):#按下 's' 顯示中央 HSV 值並辨識顏色範圍
91         #獲取中央像素的 HSV 值
92         hsv_center=hsv_image[center_y,center_x]
93         hue,saturation,value=hsv_center
94         #終端機顯示 HSV 值
95         print(f"中央點的 HSV 值 :H={hue},S={saturation},V={value}")
96
97         #建立不同範圍的遮罩
98         mask_light=cv.inRange(hsv_image,lower_light,upper_light)
99         mask_medium=cv.inRange(hsv_image,lower_medium,upper_medium)
100        mask_dark=cv.inRange(hsv_image,lower_dark,upper_dark)

```

取得影像正中央之 HSV 值並顯示

建立顏色範圍的遮罩

1. lower_light 和 upper_light：定義淺色咖啡豆的 HSV 範圍。範圍內的像素將被標記為「白色」，不符合標記為「黑色」。用於後續計算數量，並歸類顏色使用。
2. mask_light、mask_medium、mask_dark：分別對應淺色、中等顏色和深色的遮罩，代表符合該範圍的像素集合。

計算符合遮罩範圍的像素數量

3. cv.countNonZero()：計算遮罩中非零（白色像素）的數量。非零像素表示圖像中符合該顏色範圍的點數。
4. 意義：比較 count_light、count_medium 和 count_dark，可以得出咖啡豆影像中哪一種顏色範圍佔據的像素最多。

```

101        #計算每個遮罩的白色像素（表示匹配的範圍大小）
102        count_light=cv.countNonZero(mask_light)
103        count_medium=cv.countNonZero(mask_medium)
104        count_dark=cv.countNonZero(mask_dark)
105
106        #判斷哪個範圍最大
107        if count_light>count_medium and count_light>count_dark:
108            print("偵測結果：淺色咖啡豆")
109        elif count_medium>count_light and count_medium>count_dark:
110            print("偵測結果：中等顏色咖啡豆")
111        elif count_dark>count_light and count_dark>count_medium:
112            print("偵測結果：深色咖啡豆")
113        else:
114            print("無法確定顏色（可能顏色區間需要調整）")
115    cv.destroyAllWindows()

```

比較三個顏色範圍內的像素數量

1. 如果 count_light 最大，說明淺色像素最多，推測主要是淺色咖啡豆。
2. 如果 count_medium 最大，說明中等顏色像素最多，推測主要是中等顏色咖啡豆。
3. 如果 count_dark 最大，說明深色像素最多，推測主要是深色咖啡豆。
4. 如果三種顏色的像素數量接近，可能顏色區間的設定不夠準確，需要調整。

- **BGR to HSV**：RGB 可以透過計算轉換成 HSV，為了方便計算，需要先將 r、g、b 這三個值做正規化，讓值域分布在 0~1 之間。計算方式如下。

$$\begin{aligned}
 r, g, b &\in [0, 1] \\
 h &\in [0^\circ, 360^\circ] \quad \max = \max(r, g, b) \\
 s, v &\in [0, 1] \quad \min = \min(r, g, b)
 \end{aligned}$$

$$h = \begin{cases} 0^\circ & \text{if } \max = \min \\ 60^\circ \times \frac{g-b}{\max-\min} + 0^\circ, & \text{if } \max = r \text{ and } g \geq b \\ 60^\circ \times \frac{g-b}{\max-\min} + 360^\circ, & \text{if } \max = r \text{ and } g < b \\ 60^\circ \times \frac{b-r}{\max-\min} + 120^\circ, & \text{if } \max = g \\ 60^\circ \times \frac{r-g}{\max-\min} + 240^\circ, & \text{if } \max = b \end{cases}$$

$$s = \begin{cases} 0, & \text{if } \max = 0 \\ \frac{\max-\min}{\max} = 1 - \frac{\min}{\max}, & \text{else} \end{cases}$$

$$v = \max$$

- **高斯模糊 (Gaussian Blur)**：

高斯模糊是一種平滑影像的濾波技術，以高斯函數作為基礎，主要用於去除圖像中的噪聲與細節。數學公式如下(two dimension)：

$$G(u, v) = \frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/(2\sigma^2)}$$

r 是模糊半徑， σ 是常態分布的標準偏差。在二維空間中，高斯模糊的公式生成中心為峰值的常態分布，同心圓等高線的曲面。模糊是通過高斯卷積矩陣進行，矩陣內像素的權重隨距離中心像素的增加而降低。這種處理方式較均值模糊濾波器更能保留邊緣效果，因為高斯分布賦予中心像素最大的權重，保留了更多細節資訊。

- **高斯核 (Gaussian Kernel)**：基於高斯分佈（正態分佈）來生成的。這意味著核的數值會根據距離中央像素的遠近來設置，距離越近的像素權重越大，距離越遠的像素權重越小。這樣的設計是為了實現一種「平滑」效果，使得影像中的細節變得更加模糊。
- **核的大小 (Kernel Size)** 是指核矩陣的維度，通常是奇數，例如 3×3, 5×5, 7×7。
 - 小的核（例如 5×5）會產生較輕微的模糊效果，保留更多細節。
 - 大的核（例如 25×25）會產生較強的模糊效果，使更多的細節被平滑掉。
 - 因為大核會考慮更多周圍的像素，因此每個像素的模糊程度更高(見圖)。

- **實際應用中核對高斯模糊的影響：**

高斯模糊的過程就是將**高斯核與影像進行「卷積」**，這是數學上對影像的加權平均操作。核會在影像上滑動，對每個像素及其周圍的像素進行加權平均。這樣的操作會使影像變得模糊。假設使用 5×5 的高斯核，**並對影像進行高斯模糊。對每個像素，會取它周圍 5×5 範圍內的像素**，根據它們的高斯權重來計算新值。會讓原本較尖銳的邊緣變得平滑。

以下的圖片用以解釋高斯模糊的效果，並在原圖加上雜訊模擬，以凸顯高斯模糊效果。sensor 在 sensing 的過程中，突然被一個脈波干擾到，很可能就會產生所謂的 Impulse noise(見圖)。並可看出當核的大小提高，雜訊濾除效果更佳，但相對的會使畫面更為模糊。

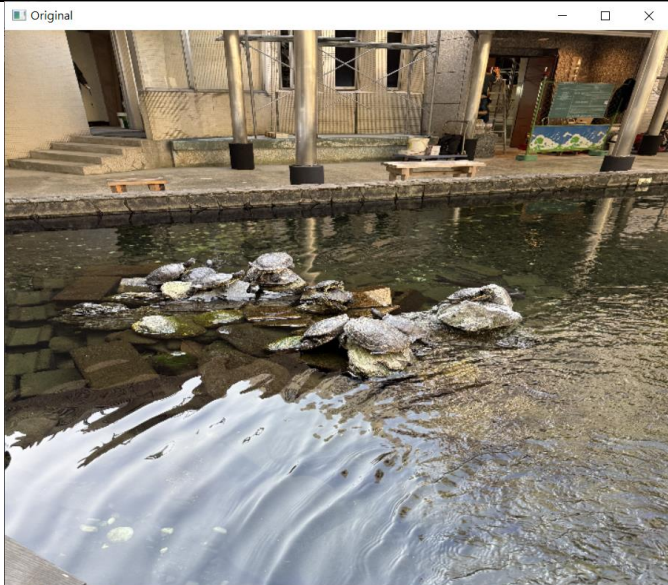


圖 4.4 照片原圖

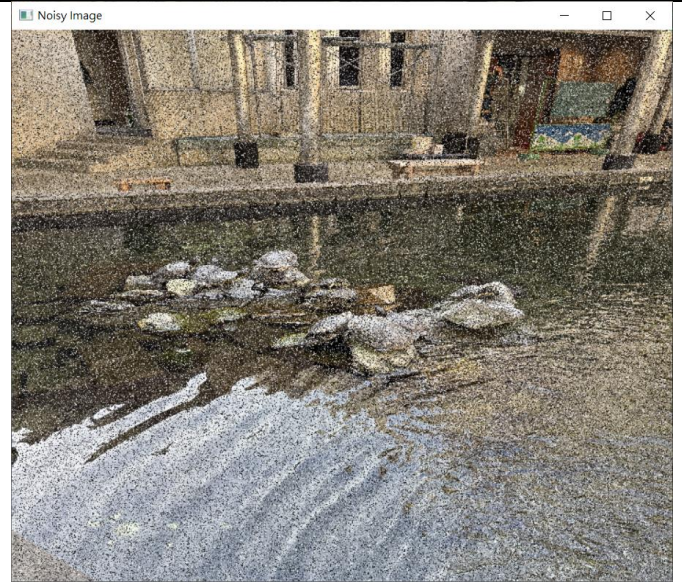


圖 4.5 加入 Impulse noise

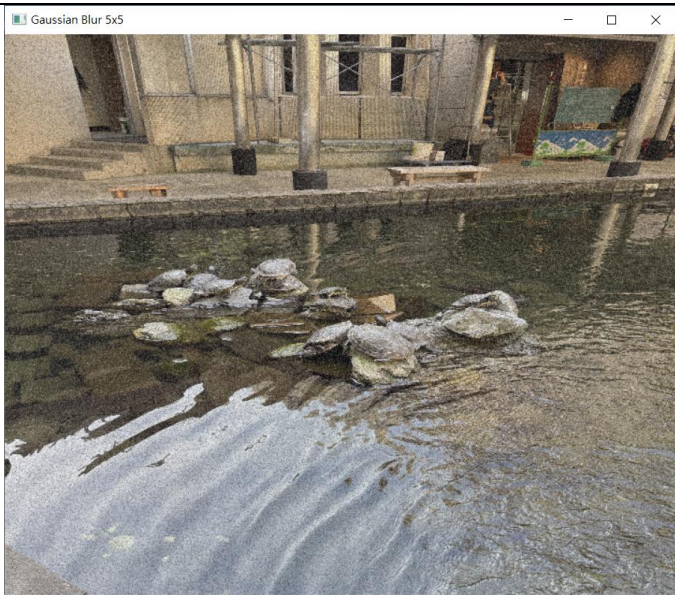


圖 4.6 高斯模糊_5×5 高斯核

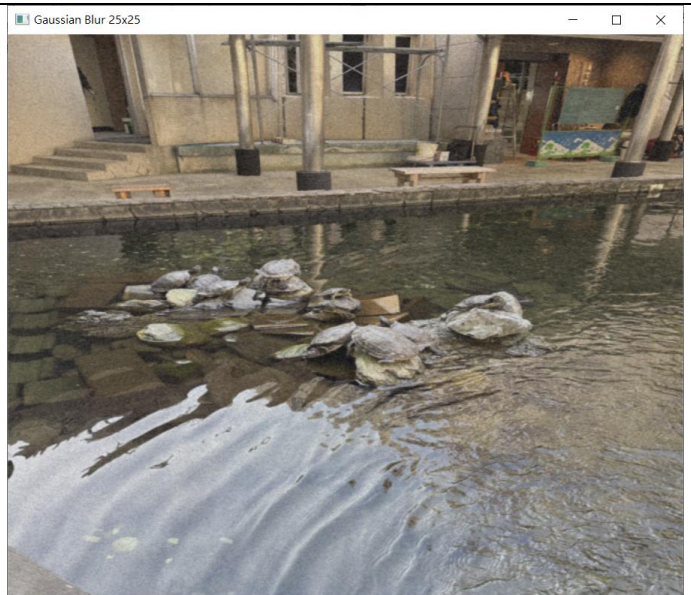
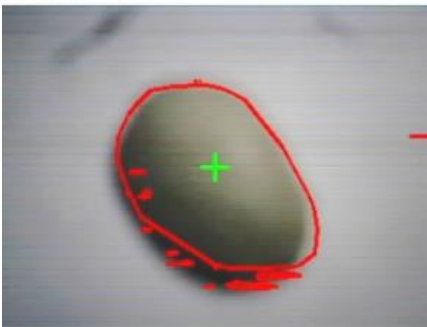
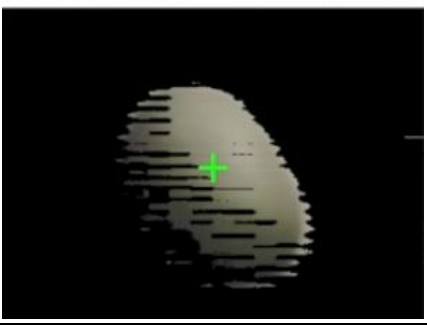
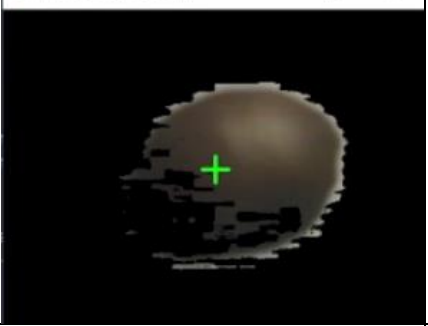
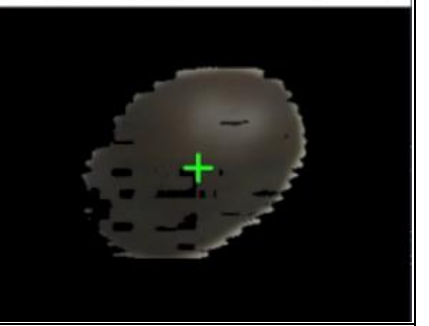
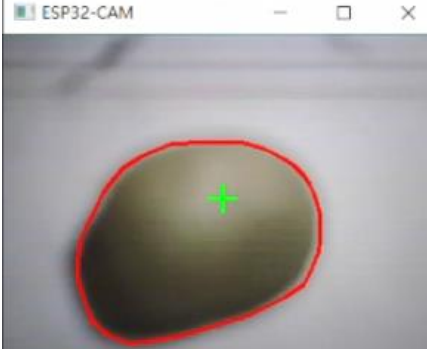
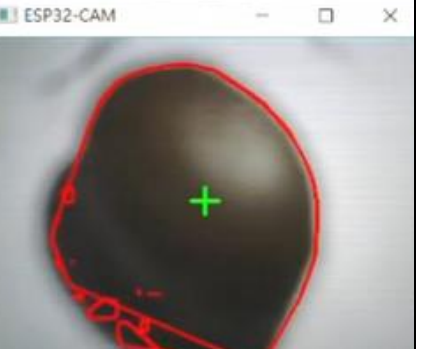
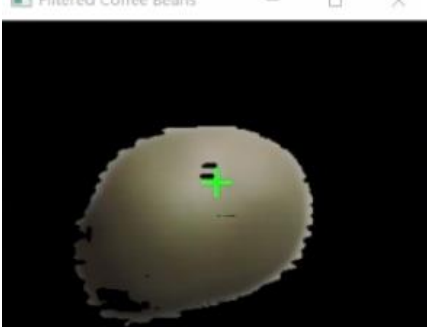
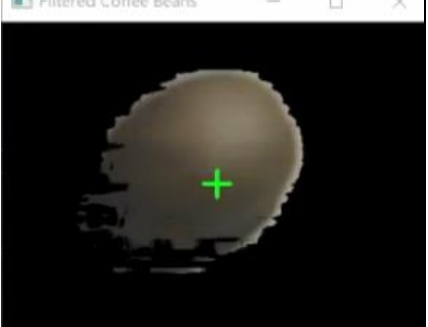


圖 4.7 高斯模糊_25×25 高斯核

比較圖 4.6 與圖 4.7 可見對於較高的核數，
濾除雜訊的效果更加，但相對的也使圖片更加模糊。

	淺色	中等	深色
原始影像			
濾除背景			
輸出	中央點的 HSV 值：H=30, S=59, V=121 偵測結果：淺色咖啡豆	中央點的 HSV 值：H=11, S=91, V=45 偵測結果：中等顏色咖啡豆	中央點的 HSV 值：H=19, S=62, V=33 偵測結果：深色咖啡豆
原始影像			
濾除背景			
輸出	中央點的 HSV 值：H=22, S=37, V=153 偵測結果：淺色咖啡豆	中央點的 HSV 值：H=18, S=81, V=63 偵測結果：中等顏色咖啡豆	中央點的 HSV 值：H=18, S=104, V=54 偵測結果：深色咖啡豆
H S V 區 間	[10-30,30-150,90-200]	[10-50,50-200,40-80]	[5-35,20-180,30-50]